Project 1: 3D Perception

Alessandro La Mantia

1. Title and Project Overview

3D Perception with PointNet: From ModelNet40 Classification to Fine-Tuning on Horses

- Environment and Tools: VS Code, Python, PyTorch, Streamlit, Plotly.
- **Key Objectives:**
  - Understand and process unstructured 3D data (Point Clouds) using the PointNet neural network architecture.
  - Develop an interactive interface for exploring and inspecting the internal mechanisms of the network.
  - Conduct a fine-tuning experiment on a new unseen class (horse) and critically analyze the trade-off with existing classes.

2. The Learning Process

Work Methodology and Incremental Approach

- Phase 1 — Theoretical Study: Understanding the challenges of 3D geometry (permutation invariance, rotation invariance) and the PointNet architecture.
- Phase 2 — Inference Pipeline and Inspection: Building Parquet data reading, preprocessing, and internal feature visualization.
- Phase 3 — Interactive App Development: Creating the Streamlit/Plotly dashboard to make theoretical concepts visible.
- Phase 4 — Critical Experimentation: Designing the adaptation experiment for a new class (horses) and quantitatively evaluating the accuracy trade-off.

3. 3D Data: Point Clouds and the ModelNet40 Dataset

Direct Geometric Representation

- Point Cloud: An unordered set of 3D points defined by (X, Y, Z) coordinates. Preserves continuous geometry without the complexity of 3D grids (Voxels) or 2D projections.
- **ModelNet40 Dataset: The ModelNet40 dataset contains 12,308 CAD objects, divided into 40 categories:**
  - 9,840 training objects.
  - 2,468 test objects.
- Main Challenge: The order of points in the file is arbitrary; the network must produce the same result regardless of how the points are ordered.

4. Essential Geometric Preprocessing

Sampling, Centering, and Normalization

- Uniform Sampling: Reduction to 1,024 points per sample to ensure uniform input and computational efficiency.

- Spatial Normalization: Moving the center of the shape to the origin and scaling it to fit within a unit-radius sphere.

```
def prepara_punti(points, num_punti=1024, seed=42):
    # Random generator with a fixed seed: always the same sampling
    rng = np.random.default_rng(seed)

    # Randomly select num_punti points from the point cloud without repetition
    indices = rng.choice(len(points), num_punti, replace=False)
    selected = points[indices]

    # Shift the point cloud so that its center coincides with the origin
    selected = selected - selected.mean(axis=0)

    # Compute the maximum distance of the points from the origin
    radius = np.linalg.norm(selected, axis=1).max()

    # Normalize
    return selected / radius if radius > 0 else selected
```

5. PointNet Architecture: Key Concepts

How PointNet Handles Point Order and Transformation

- 1. T-Net Modules (Spatial Transformers)
  - Learned transformation: computes 3×3 and 64×64 matrices to transform coordinates and features.
  - Adaptation: can rotate, stretch, or compress the point cloud differently for each object.
  - Objective: obtain a more effective representation for classification.
- 2. Feature Extraction
  - Point-by-point processing: PointNet applies the same operations to each point independently.
  - Learned features: each point is represented by 1,024 numerical features.
- **3. Global Max Pooling**
  - For each feature, the maximum value across all 1,024 points is selected.
  - The result is a single global vector of 1,024 values representing the entire object.

6. PointNet Architecture: Essential Structure

From a Single Point to the Global Descriptor

```
# Conceptual structure of the Forward pass in PointNet
def forward(self, x):
    # x has shape: (Batch, 3, N_punti)
    trans_3x3 = self.input_transform(x)          # Spatial T-Net (3x3)
    x = torch.bmm(trans_3x3, x)                  # Geometric alignment
    x = F.relu(self.conv_features(x))             # Local feature extraction (64)

    trans_64x64 = self.feature_transform(x)      # Feature T-Net (64x64)
    x = torch.bmm(trans_64x64, x)
    x = F.relu(self.conv_high_dim(x))            # High-dimensional mapping (1024)

    global_descriptor = torch.max(x, dim=2)[0]   # GLOBAL MAX POOLING (Permutation invariance)
    logits = self.classifier_mlp(global_descriptor) # Final classification (40 classes)
    return logits
```

7. Internal Inspection with PyTorch Forward Hooks

From Max Pooling to Classification

- 1. Which points contribute to the descriptor?
  Forward Hooks allow us to capture the activations of the 1,024 channels before Global Max Pooling. For each channel, we can identify the point that produced the maximum value.
- 2. What do the colors indicate?
  The color represents how many channels selected that point as their maximum. A high value indicates that the point contributes to the maximum of more features, but does not represent an importance percentage or automatically identify a specific part of the object.
- 3. From descriptor to classification
  Global Max Pooling produces a global descriptor of 1,024 values. The final layers process this vector and produce 40 scores, one for each class. The class with the highest score determines the final prediction.

8. Main Interactive Application

3D Exploration and Visualization (Streamlit & Plotly)

- **User Interface (localhost:8501):**
  - Real-time 3D rendering of point clouds that can be manipulated and rotated.
  - Visualization of classification probabilities (Softmax) for the Top-5 categories.
  - Interactive inspection of Max Pooling activations with color mapping overlaid on the 3D model.

9. Difficulties and Solutions

The Main Technical Challenges of the Project

💾 Data management

Very large 3D dataset → read data only when needed, avoiding loading everything into RAM.

🎲 Random sampling

Different samples could change the prediction → use a fixed seed to obtain reproducible results.

🔄 Point order invariance

Verify that reordering points does not change the prediction → automated tests on the model.

Fine Tuning

11. Domain Extension: The "Horse" Class

Motivation and Research Question

- The Context: The original ModelNet40 dataset does not include the "horse" category.
- **Strategic Research Question:**

  "How many samples of the new class are needed to teach it to the model, and what is the cost in terms of performance degradation on the original 40 classes?"

- Conceptual Approach: Evaluate Continual Learning capabilities and the prevention of Catastrophic Forgetting.

12. Dataset Preparation and Replay Strategy

Extraction from Objaverse and Forgetting Prevention

- New Class Source: 3D horse models extracted from Objaverse (LVIS dataset) and sampled at 2,048 points.

- Rigorous Deduplication: Checking and removing duplicates through geometric mesh hashing.
- **Replay Technique (Memory Buffer):**
  - To prevent the network from "forgetting" the 40 original classes, 5 samples for each old class were included in the fine-tuning training set (200 replay samples in total).
- Fixed Evaluation Test Set: 420 objects across classes (400 objects from the old classes + 20 unseen horses).

13. Fine-Tuning Strategy: Head Adaptation

Freezing the Feature Backbone

- Backbone Freezing: The weights of the feature extraction modules (T-Net, Convolutions, Max Pooling) are frozen. The fundamental geometric representations remain unchanged.
- **Modifying the Last Layer:**
  - The final classifier goes from 40 to 41 outputs.
  - The first 40 weights are initialized from the pre-trained weights; the 41st neuron is initialized for the horse class.
- Computational Advantage: Extremely fast training, executable in a few seconds even on a CPU.

14. Fine-Tuning Implementation

Head Adaptation with PyTorch

```
def fit_head(original, support, replay, replay_y, seed, steps, lr):
    torch.manual_seed(seed)

    # New classifier: 40 original categories + horse.
    head = nn.Linear(256, 41)

    # Copy the weights of the already known categories.
    with torch.no_grad():
        head.weight[:40].copy_(original.weight)
        head.bias[:40].copy_(original.bias)

    optimizer = torch.optim.Adam(head.parameters(), lr=lr)
    positives = torch.full((len(support),), 40, dtype=torch.long)

    for _ in range(steps):
        optimizer.zero_grad()
        # Half the weight to horses, half to the old categories.
        loss = 0.5 * nn.functional.cross_entropy(head(support), positives)
        loss += 0.5 * nn.functional.cross_entropy(head(replay), replay_y)
        loss.backward()
        optimizer.step()

    return head.eval()
```

15. Experimental Results: The Trade-Off

Quantitative Analysis of the Metrics

- With 1–5 Horses: The network struggles to generalize. Horse Recall is low (< 40%).
- With 10–25 Horses: An excellent balance is achieved: Horse Recall around 80–85% and minimal impact on the old classes.
- With 50 Horses: Horse Recall rises to 95%, but Precision decreases and accuracy on the original 40 classes drops.

> Deduction: Increasing the data for the new class without increasing replay samples leads the network to develop a bias toward the horse class, mistaking complex objects from old classes for horses.

16. Second App: Experiment Dashboard

Comparative Analysis of Results (Streamlit)

- **Experiment Interface (localhost:8502):**
  - Interactive charts showing Recall, Precision, and Accuracy as the data budget varies.
  - Visual inspection of correctly or incorrectly classified test horses.
  - Direct comparison of predictions from models trained with 1 vs 50 horses.

17. Critical Analysis: Failures and Lessons Learned

What Did Not Work and How It Was Resolved

Initial Loss Imbalance:
In the initial trials without balancing the Loss weights, the network completely ignored the single horse presented (Recall = 0%) because the 40 classes dominated the gradient. This was resolved by introducing Weighted Cross-Entropy Loss.

The Illusion of High Recall:
Measuring only Recall for the new class gave the illusion that 50 horses were the perfect choice. Precision analysis revealed degradation on the old classes, highlighting the importance of balanced metrics.

18. Evaluation of Model and Dataset Limitations

Domain Shift and Generality Considerations

- Domain Shift Between Datasets: Horses come from Objaverse (modern mesh models), while the 40 classes come from ModelNet40 (clean synthetic CAD). The difference in point density and distribution creates a "domain gap".
- Noise in Real Data: Real point clouds (e.g., from LiDAR sensors) contain occlusions and noise that are absent from the clean synthetic data tested.
- PointNet Limitations: PointNet processes each point individually before pooling; it does not capture detailed local geometric structures like more recent architectures (e.g., PointNet++ or DGCNN).

19. What Was Accomplished

Concrete Project Results

- Point Cloud Pipeline: loading data from Parquet, sampling 1,024 points, centering and normalizing the point cloud.
- PointNet Analysis: study of T-Net, feature extraction, and Global Max Pooling, up to the global vector of 1,024 features.
- Internal Inspection: using Forward Hooks to identify which points contribute to feature maxima and visualize them on the point cloud.
- Fine-Tuning: adding the new horse class by going from 40 to 41 classes, while retaining replay of 200 samples from the original classes.
- Evaluation: comparing performance on the same test set of 420 objects to verify recognition of the new class and retention of previously learned classes.

20. Conclusions and Future Developments

Future Developments

- Test Larger Amounts of Data: repeat fine-tuning with more examples of the horse class to study how the model’s learning capacity changes.
- Improve Fine-Tuning: compare the current Head Adaptation with partial fine-tuning of the network, progressively unfreezing some backbone layers.
- Analyze Features More Thoroughly: extend the activation visualization to understand which parts of the point cloud most influence classification.
- Expand the Evaluation: repeat the experiments with more classes and more objects to verify whether the results observed for horses generalize to other cases.

Thank you for your attention